

プログラミング言語 Egison 入門

江木 聡志

2020 年 3 月 1 日

はじめに

本書は、著者が自作のプログラミング言語 Egison を通して提唱しているパターンマッチ指向プログラミングという新しいプログラミング・パラダイムをできるだけコンパクトに紹介することを目的として執筆された。パターンマッチまわりの Egison の構文の紹介と、それらの構文を使ったプログラミングテクニックの紹介を軸として執筆した。

Egison はより直感的にアルゴリズムを表現したいという動機で、主に著者自身により発案されたプログラミング言語のアイデアを実証するためにつくられた proof of concept のプログラミング言語である。Egison に実装されているこれらのアイデアのうち、もっとも重要な機能は本書の主題であるパターンマッチである。Egison のパターンマッチの特徴は、ユーザーにより拡張可能でかつ、非線形パターン（パターン変数に束縛した値を同じパターン内で参照するパターン）をバックトラッキングにより効率的に処理することである。本書の目的は、この機能がいかに表現力が豊かで、応用範囲がひろく、将来性が高いのか読者に著者と同じくらいに感じてもらうことである。本書ではふれないが、他の重要な機能には、微分幾何の計算を表現するのに便利なテンソルの添字記法をプログラム中で使うための機能がある。

本書内のサンプルプログラムは、現在の版の Egison の文法を使って記述されている。以降の開発で Egison の文法が変わる可能性が高い。しかし、サンプルプログラムの裏のアイデアの根幹は普遍的なものであるので、安心して読んでほしい。

本書がきっかけとなって、プログラミング言語、とくにパターンマッチの研究をする研究仲間が読者の中から現れてくれたらとてもうれしい。

2020 年 1 月 6 日

江木 聡志

目次

はじめに	ii
第 1 章 プログラミング Egison とは	1
1 プログラミング言語全体の中での Egison の位置づけ	1
2 パターンマッチ指向プログラミングとは	2
3 本書の構成	3
第 2 章 Egison 早巡り	4
1 matchAll式によるパターンマッチ	4
2 値パターンと述語パターンによる非線形パターンの表現	5
3 バックトラッキングによる効率的な非線形パターンマッチ	6
4 マッチャーによるパターンの拡張性と多相性	6
5 matchAllDFS式によるパターンマッチ結果の順序の制御	7
6 and パターン・or パターン・not パターン	8
7 ループパターン	8
8 シーケンシャルパターン	9
9 forall パターン	10
10 パターン関数によるパターンのモジュール化	11
11 マッチャー合成による新しいマッチャーの生成	11
第 3 章 パターンマッチ指向プログラミング入門	13
1 リストのパターンマッチ	13
2 多重集合のパターンマッチ	13
3 複数のコレクションの比較	13
4 ループパターンを使う	13
第 4 章 実践パターンマッチ指向プログラミング	14
1 リストプログラミング	14
2 ポーカーの役判定	14
3 ゲーム木のパターンマッチ	14

4	N クイーン問題	14
5	SAT ソルバー	14
第 5 章	マッチャーを定義しよう	15
1	Egison 内部のパターンマッチアルゴリズム	15
2	unorderedPair マッチャーの定義	15
3	multiset マッチャーの定義	15
4	soretedList マッチャーの定義	15
第 6 章	Egison パターンマッチの実装	16
1	インタプリタ実装	16
2	ほかの関数型言語へのトランスレータ実装	16
3	今後の研究の展開	16
あとがき		17
参考文献		18
索引		19

第 1 章

プログラミング Egison とは

本章では、Egison がどのような言語であるのかを紹介する。本章の内容は、いま完全に理解する必要はない。Egison とそれが提唱しているパターンマッチ指向プログラミングがどのような点で優れているのかその大体のイメージをつかんでもらいたい。

1 プログラミング言語全体の中での Egison の位置づけ

数多あるプログラミング言語のなかでの Egison の著しい特徴は、世界で初めて自身に実装されたアルゴリズムの記述を直感的にするための機能を有しているところである。アルゴリズムの記述を本質的に簡潔にするプログラミング言語のアイデアは、歴史を紐解いてみても、そう多くはない。そういった意味で Egison は数多にあるプログラミング言語のなかでも希少な言語であるだろう。

プログラミング言語の機能には大きく分けて 2 種類ある。コンピューターを含むさまざまな機械の制御を簡潔に記述するための機能と、数学的なアルゴリズムを簡潔に記述するための機能である。前者の機能は、たとえば、オペレーティング・システムやウェブ・ブラウザなど我々が日常的に使用しているさまざまなソフトウェアを実装する上で重要な機能である。後者の機能は、(TODO:)。Egison は後者の機能の拡充に注力してつくられている。

アルゴリズムを簡潔に記述することを目指しているプログラミング言語の流派の主流は、関数型プログラミング言語である。関数型プログラミングでは、コンピューターがプログラムを実行するのはプログラムはあまり意識せず、アルゴリズムを数学的にそのまま記述することを目指す。関数型プログラミングでは、コンピューターが実行できる形にプログラムを書き直す役割は主にコンパイラが果たす。

関数型プログラミング言語によるプログラムの記述の大きな特徴は、関数を他の組み込みデータ型と同じようにプログラマが扱えることである。具体的にいうと、関数を別の関数の引数として渡したり、関数を返す関数を定義することができる。そのおかげで、関数型プログラミング言語以外の言語ではモジュール化がむずかしい処理がモジュール化できることが多々ある。

しかし、関数型プログラミング言語でも、頭のなかのアルゴリズムのイメージを、関数型プログ

ラムとして記述できるように翻訳する必要がある場合もある。(TODO:)

2 パターンマッチ指向プログラミングとは

本節では、いくつかの例をみることによって、パターンマッチ指向プログラミングとはどのようなプログラミング・パラダイムであるのかそのイメージを紹介する。

パターンマッチ指向プログラミングによってプログラムの記述が直感的になっていることわかりやすい例に `intersect` 関数の実装がある。 `intersect` は 2 つのリストを引数にとり、それらのリストに共通する要素のリストを返す関数である。 `Egison` を使って引数のリストをそれぞれ多重集合としてパターンマッチすると、2 つのリストの共通要素にマッチするパターンを記述することにより、 `intersect` を記述できる。^{*1}

```
1 intersect xs ys :=
2   matchAllDFS (xs, ys) as (multiset eq, multiset eq) with
3   | ($x :: _, #x :: _) -> x
```

対して、 `Egison` のパターンマッチを使わずに関数型プログラミングのスタイルで `Haskell` で記述すると、リストに対する関数を組み合わせて共通要素を抜き出すための方法を記述する必要がある。

```
1 intersect xs ys = filter (\x -> any (== x) ys) xs
```

このプログラムは、リスト内包表記を使って以下のように書き直すこともできる。

```
1 intersect xs ys = [x | x <- xs, any (== x) ys]
```

`Egison` のパターンマッチを使ったプログラムは、2 つのリストの共通要素にマッチするパターンを記述しているだけであるのに対し、関数型プログラミングでは、どうやってその共通要素を取り出すかその方法をプログラマが考えて記述する。前者のように「何を計算したいか (what to do)」を記述するプログラミングスタイルは宣言型プログラミング、後者のように「どのように計算するか (how to do)」を記述するプログラミングスタイルは手続き型プログラミングと呼ばれている。「何を計算したいか」から「どのように計算するか」が明らかである場合は、「何を計算したいか」を記述する宣言型プログラミングのほうがプログラムが読みやすく簡潔になる。`intersect` の例の場合は、 `Egison` が多重集合のパターンマッチアルゴリズムをモジュール化できるおかげで、宣言的なプログラミングが可能になっている。

少し毛色の違う例として、 `concat` 関数をパターンマッチ指向プログラミングで定義する。リストのリストの要素にマッチするパターンを記述することにより、 `concat` を定義できる。

```
1 concat xss :=
2   matchAllDFS xss as list (list something) with
```

^{*1} プログラムの読み方は次章以降で紹介する。

```

3 | _ ++ ( _ ++ $x :: _ ) :: _ -> x
4
5 concat [[1,2],[3],[4,5]]
6 -- [1,2,3,4,5]

```

さらにおもしろいパターンマッチ指向プログラミングの例として，unique関数を定義する．後方に自身と同じ値が含まれない要素にマッチするパターンを記述することにより uniqueを定義できる．

```

1 unique xs :=
2   matchAllDFS xs as list eq with
3   | _ ++ $x :: !( _ ++ #x :: _ ) -> x
4
5 unique [1,2,3,2,4]
6 -- [1,3,2,4]

```

次章以降で，上記のプログラムで使われている Egison の構文や，機能，プログラミングテクニックの解説をしていく．

3 本書の構成

本書は可能な限りコンパクトに，パターンマッチ指向プログラミングとそのための Egison の機能の解説をまとめることを目指した．そのため，本書は前提知識として，関数型プログラミング（Lisp 系言語や，OCaml, Haskell を使ったプログラミング）の知識を仮定している．既存の関数型言語にも存在する機能については，独立した節を設けず，登場したときに軽く解説するにとどめた．

第2章では，パターンマッチに関する Egison の構文を一通り紹介する．第3章では，Egison のパターンマッチを活かしたプログラミングスタイルであるパターンマッチ指向プログラミングとはなにかを紹介する．第4章では，より実践的なパターンマッチ指向プログラミングの例を紹介する．第5章では，新しいマッチャーの定義の方法を説明する．第6章では，Egison のパターンマッチの実装について紹介する．

第 2 章

Egison 早巡り

本章は、パターンマッチ指向プログラミングのための Egison の機能を一通り紹介する。

1 matchAll 式によるパターンマッチ

パターンマッチを記述するためのいくつかの構文を Egison は提供している。そのうちもっとも基本的な構文は matchAll 式である。

```
1 matchAll [1,2,3] as list something with
2 | $x :: $ts -> (x, ts)
3 -- [(1,[2,3])]
```

matchAll 式は、ターゲット（上記の場合、`[1,2,3]`）、マッチャー（上記の場合、`list something`）、マッチ節（上記の場合、`$x :: $xs -> (x,xs)`）から構成される。マッチ節は、パターン（上記の場合、`$x :: $xs`）とボディ（上記の場合、`(x,xs)`）からなる。matchAll 式は、既存のプログラミング言語のマッチ式と同様に、ターゲットとパターンのパターンマッチを試みて、もしマッチしたらそのマッチ節のボディを評価する。

Egison の matchAll 式の特徴は、(1) matchAll という名前と結果としてリストを返すことと、(2) マッチャーという追加の引数をとることにある。(1) の特徴は、複数の結果をもつパターンマッチをサポートするための特徴である。matchAll 式は複数のパターンマッチの結果すべてについてボディを評価して、その結果を集めてリストとして返す。上記の例のパターンに使われている `::` はコンスパターンと呼ばれるリストを先頭の要素と残りのリストに分解するパターンコンストラクタである。そのため、上記の例の場合は、パターンマッチの結果が一つであるので、長さが 1 のリストを返している。(2) の特徴は、パターンマッチアルゴリズムの拡張性とパターンの多相性を実現するための特徴である。マッチャーは Egison 以外の言語ではみられない独自のオブジェクトで、パターンマッチアルゴリズムを保持するためのオブジェクトである。マッチャーによるパターンの多相性については、4 節で扱う。

-- は Egison のインライン・コメント・デリミタである。本書を通して、プログラム直後のイン

ライン・コメントを使って、プログラムの実行結果を記述する。

```
1 matchAll [1,2,3] as list something with
2 | $hs ++ $ts -> (hs, ts)
3 -- [([], [1, 2, 3]), ([1], [2, 3]), ([1, 2], [3]), ([1, 2, 3], [])]
```

```
1 matchAll ターゲット as マッチャー with
2 | パターン -> ボディ
3 | パターン -> ボディ
4 ...
```

マッチ節が1つであるときは、下記のようにマッチ節の前の|を省略することができる。

```
1 matchAll ターゲット as マッチャー with パターン -> ボディ
```

2 値パターンと述語パターンによる非線形パターンの表現

matchAll式は、非線形パターンと組み合わせたときにその本領を発揮する。たとえば、以下の非線形パターンは、対象のコレクションが同値な要素のペアを含む場合にパターンマッチに成功する。

```
1 matchAll [1,2,3,2,4,3] as list integer with
2 | _ ++ $x :: _ ++ #x :: _ -> x
3 -- [2,3]
```

値パターンは非線形パターンを表現するために重要な役割を果たす。値パターンは、値パターンの中身とターゲットが等しいかどうかチェックする。値パターンの先頭には、#が付加される。#の後ろには任意の式を書くことができる。#に続く式は、その値パターンの左側に現れたパターン変数に束縛された値を参照しながら評価される。この動作を実現するため、Egisonのパターンは左から右に順番に処理されることが決まっている。その結果、 $x :: \#x :: _$ のようなパターンは妥当であるが、 $\#x :: x :: _$ は間違いである。

よりおもしろい非線形パターンの例として、双子素数のパターンマッチを紹介する。双子素数とは、 $(p, p+2)$ という形の素数のペアのことをいう。primesには素数の無限列が束縛されている。primesはEgisonの組み込みライブラリで定義されている。このmatchAll式は、素数の無限列からすべての双子素数を順番に抜き出す。

```
1 twinPrimes := matchAll primes as list integer with
2 | _ ++ $p :: #(p + 2) :: _ -> (p, p + 2)
3
4 take 8 twinPrimes
5 -- [(3, 5), (5, 7), (11, 13), (17, 19), (29, 31), (41, 43), (59, 61), (71, 73)]
```

パターンマッチのときに、同値性よりも一般的な述語を使いたいことがある。述語パターンは、そのために用意されている組み込みパターンである。述語パターンは、述語パターンの中身の述語にターゲットを適用したときに真が帰ってきた場合、パターンマッチに成功するパターンである。述語パターンの先頭は、?からはじまり、?のあとには 1 引数の述語が続く。

```
1 twinPrimes = matchAll primes as list integer with
2   | _ ++ $p : ?(\q -> q == p + 2) : _ -> (p, p + 2)
```

3 バックトラッキングによる効率的な非線形パターンマッチ

The pattern-matching algorithm inside Egison includes the backtracking mechanism for efficient non-linear pattern matching.^{*1}

```
1 matchAll [1..n] as list integer with _ ++ $x :: _ ++ #x :: _ -> x
2 -- O(n^2) で [] を返す
3 matchAll [1..n] as list integer with _ ++ $x :: _ ++ #x :: _ ++ #x :: _ -> x
4 -- O(n^2) で [] を返す
```

The above expressions match a collection that consists of integers from 1 to n as a list of integers for enumerating identical pairs and triples, respectively. This target collection contains neither identical pairs nor triples, therefore both expressions return an empty collection.

When evaluating the second expression, Egison interpreter does not try pattern matching for the second $\#x$ because pattern matching for the first $\#x$ always fails. Therefore, the time complexities of the above expressions are identical. The pattern-matching algorithm inside Egison is discussed in [3] in detail.

4 マッチャーによるパターンの拡張性と多相性

Another merit of matchers, in addition to the extensibility of pattern-matching algorithms, is the ad-hoc polymorphism of patterns. The ad-hoc polymorphism of patterns is important for non-free data types because some data are pattern-matched as various non-free data types at the different parts of a program. For example, a collection is pattern-matched as a list, a multiset, and a set. Polymorphic patterns reduce the number of names for pattern constructors.

In the following sample, a list $[1,2,3]$ is pattern-matched using different matchers with the same cons pattern. In the case of sets, the rest elements are the same as the original

^{*1} The Author's paper [2] on Scheme macros that implement our pattern-matching system discusses the performance of a program in pattern-match-oriented style. A program in pattern-match-oriented style is currently two times slower when compared with the same program in functional programming style.

collection because we ignore the redundant elements. If we regard a set as a collection that contains infinitely many copies of each element, this specification of the cons pattern for sets is natural.

```
1 matchAll [1,2,3] as list something with $x :: $xs -> (x,xs)
2 -- [(1,[2,3])]
3 matchAll [1,2,3] as multiset something with $x :: $xs -> (x,xs)
4 -- [(1,[2,3]),(2,[1,3]),(3,[1,2])]
5 matchAll [1,2,3] as set something with $x :: $xs -> (x,xs)
6 -- [(1,[1,2,3]),(2,[1,2,3]),(3,[1,2,3])]
```

Polymorphic patterns are useful especially for value patterns. As well as other patterns, the behavior of value patterns is dependent on matchers. For example, an equality $[1,2,3] \hookrightarrow == [2,1,3]$ between collections is false if we regard them as lists but true if we regard them as multisets. Still, thanks to ad-hoc polymorphism of patterns, we can use the same syntax for both types. This dramatically improves the readability of the program and makes programming with non-free data types easy.

```
1 matchAll [1,2,3] as list integer with #[2,1,3] -> "Matched" -- []
2 matchAll [1,2,3] as multiset integer with #[2,1,3] -> "Matched" -- ["Matched"]
```

5 matchAllDFS式によるパターンマッチ結果の順序の制御

The `matchAll` expression is designed to enumerate all countably infinite pattern-matching results. For this purpose, users sometimes need to care about the order of pattern-matching results.

Let us start by showing a representative sample. The `matchAll` expression below enumerates all pairs of natural numbers. We extract the first 8 elements with the `take` function. `matchAll` traverses the reduction tree of pattern matching in breadth-first search to traverse all the nodes (Sect. 5.2 of [3]). As a result, the order of the pattern-matching results is as follows.

```
1 take 8 (matchAll [1..] as set something with $x : $y : _ -> (x,y))
2 -- [(1,1),(1,2),(2,1),(1,3),(2,2),(3,1),(2,3),(3,2)]
```

The above order is preferable for traversing an infinitely large reduction tree. However, sometimes, this order is not preferable (see Sect. ?? and ??). `matchAllDFS` that traverses a reduction tree in depth-first order is provided for this reason.

```
1 take 8 (matchAllDFS [1..] as set something with $x : $y : _ -> (x,y))
2 -- [(1,1),(1,2),(1,3),(1,4),(1,5),(1,6),(1,7),(1,8)]
```

6 and パターン・or パターン・not パターン

The situations where and-patterns and or-patterns are useful are similar to those of the existing languages, whereas not-patterns become useful when they are combined with non-linear pattern matching with backtracking.

We start by showing pattern matching for *prime triples* as an example of and-patterns and or-patterns. A prime triple is a triple of primes whose form is $(p, p+2, p+6)$ or $(p, p+4, p+6)$. The and-pattern is used as an as-pattern. The or-pattern is used to match both of $p+2$ and $p+4$.

```
1 primeTriples = matchAll primes as list integer with
2     _ ++ $p : (and (or #(p + 2) #(p + 4)) $m) : #(p + 6) : _ -> (p, m, p +
3     ↪ 6)
4 take 8 primeTriples -- [(5,7,11),(7,11,13),(11,13,17),(13,17,19),(17,19,23),(37,41,43)
5     ↪ ,(41,43,47),(67,71,73)]
```

A not-pattern matches a target if the pattern does not match the target, as its name implies. A not-pattern is prepended with `!`, and a pattern follows after `!`. The following `matchAll` enumerates sequential pairs of prime numbers that are *not* twin primes.

```
1 take 10 (matchAll primes as list integer with _ ++ $p : (and !#(p + 2) $q) : _ -> (p, q))
2 -- [(2,3),(7,11),(13,17),(19,23),(23,29),(31,37),(37,41),(43,47),(47,53),(53,59)]
```

7 ループパターン

A loop pattern is a pattern construct for representing a pattern that repeats multiple times. It is an extension of Kleene star operator of regular expressions for general non-free data types [1].

Let us start by considering pattern matching for enumerating all combinations of two elements from a target collection. It can be written using `matchAll` as follows.

```
1 comb2 xs = matchAll xs as list something with _ ++ $x_1 : _ ++ $x_2 : _ -> [x_1, x_2]
2
3 comb2 [1,2,3,4] -- [[1,2],[1,3],[2,3],[1,4],[2,4],[3,4]]
```

Egison allows users to append indices to a pattern variable as `$x_1` and `$x_2` in the above sample. They are called *indexed variables* and represent x_1 and x_2 in mathematical expressions. The expression after `_` must be evaluated to an integer and is called an *index*. We can append as many indices as we want like `x_i_j_k`. When a value is bound to an indexed

pattern variable `$x_i`, the system initiates an abstract map consisting of key-value pairs if `x` is not bound to a map, and bind it to `x`. If `x` is already bound to a map, a new key-value pair is added to this map.

Now, we generalize `comb2`. The loop patterns can be used for that purpose.

```

1 comb n xs = matchAll xs as list something with
2     loop $i (1,n)
3         (_ ++ $x_i : ...)
4         _ -> map (\i -> x_i) [1..n]
5
6 comb 2 [1,2,3,4] -- [[1,2],[1,3],[2,3],[1,4],[2,4],[3,4]]
7 comb 3 [1,2,3,4] -- [[1,2,3],[1,2,4],[1,3,4],[2,3,4]]

```

The loop pattern takes an *index variable*, *index range*, *repeat pattern*, and *end pattern* as arguments. An index variable is a variable to hold the current repeat count. An index range specifies the range where the index variable moves. A repeat pattern is a pattern repeated when the index variable is in the index range. An end pattern is a pattern expanded when the index variable gets out of the index range.

Inside loop patterns, we can use the *ellipsis pattern* `(...)`. The repeat pattern or the end pattern is expanded at the location of the ellipsis pattern. The repeat pattern is expanded replacing the ellipsis pattern incrementing the value of the index variable.

The repeat counts of the loop patterns in the above samples are constants. However, we can also write a loop pattern whose repeat count varies depending on the target by specifying a pattern instead of an integer as the end number. When an end number is a pattern, the ellipsis pattern is replaced with both the repeat pattern and the end pattern, and the repeat count when the ellipsis pattern is replaced with the end pattern is pattern-matched with that pattern. The following loop pattern enumerates all initial prefixes of the target collection.

```

1 matchAll [1,2,3,4] as list something with loop $i (1, $n) ($x_i : ...) _ -> map (\i ->
2     ↪ x_i) [1..n]
3 -- [[],[1],[1,2],[1,2,3],[1,2,3,4]]

```

Loop patterns are heavily used especially for trees and graphs. We work on pattern matching for trees in Sect. ?? . More formal specification of syntax and semantics of loop patterns is shown in the author's previous paper [1].

8 シーケンシャルパターン

The pattern-matching system of Egison processes patterns from left to right in order. However, there are cases where we want to change this order, for example, to refer to the value

bound to the right side of a pattern. Sequential patterns are provided for such a purpose.

Sequential patterns allow users to control the order of the pattern-matching process. A sequential pattern is represented as a list of patterns. Pattern matching is executed for each pattern in order. In the following sample, the target list is pattern-matched from the third, first, and second element in order.

```
1 matchAll [2,3,1,4,5] as list integer with
2   [ @ :      @      : $x : _,
3     (#(x + 1), @),
4     #(x + 2)] -> "Matched" -- ["Matched"]
```

@ that appears in a sequential pattern is called *later pattern variable*. The target data bound to later pattern variables are pattern-matched in the next sequence. When multiple later pattern variables appear, they are pattern-matched as a tuple in the next sequence. It allows us to apply not-patterns for different parts of a pattern at the same time as we will see in Sect ??.

Some readers might wonder that a sequential pattern can be transformed into a nested match-all expression. There are at least two reasons why it is impossible. First, a nested match-all expression breaks breadth-first search strategy: the inner match-all for the second result of the outer match-all is executed only after the inner match-all for the first result of the outer match-all is finished. Second, a later pattern variable retains the information of not only a target but also a matcher. There are cases that the matcher of match-all is a parameter passed as an argument of a function, and a pattern is polymorphic. Therefore, it is impossible to determine the matchers of inner match-all expressions syntactically.

9 forall パターン

述語パターンで奇数かどうかチェックしている。パターンマッチに成功した場合、すべてのパターンマッチ結果を返す。

```
1 matchAll [1,5,3] as multiset integer with
2 | forall ((@ & $x) :: _) ?odd? -> x
3 -- [1,5,3]
```

1 つでも第二引数のパターンのパターンマッチに失敗する第一引数のパターンマッチの結果がある場合、全体のパターンマッチに失敗する。

```
1 matchAll [1,4,3] as multiset integer with
2 | forall ((@ & $x) :: _) ?odd? -> x
3 -- []
```

not パターンでも forall パターンに近い意味のパターンを表現できる. not パターンの forall パターンに対する利点は, 以下の二点である.

- (1) forall パターン中のパターン変数に値を束縛し, 複数のパターンマッチの結果すべてを返すことができる (not パターンの中身ではパターン変数に値を束縛することができない).
- (2) forall パターンを使ったほうがより直接的に意味を表現できる場合がある.

```
1 matchAll [1,5,3] as multiset integer with
2 | !(?!odd? :: _) -> match''
3 -- [match'']
```

10 パターン関数によるパターンのモジュール化

11 マッチャー合成による新しいマッチャーの生成

Matchers are composable, and we can define matchers for such as tuples of multisets and multisets of multisets. Using this feature, we can define matchers for various data types.

First, we can define a matcher for tuples by a tuple of matchers. A tuple pattern is used for pattern matching using such a matcher. For example, we can define the intersect function using a matcher for tuples of two multisets. We work on pattern matching for tuples of collections more in Sect. ??.

```
1 intersect xs ys = matchAll (xs,ys) as (multiset eq, multiset eq) with ($x : _, #x : _) ->
    ↪ x
```

eq is a user-defined matcher for data types for which equality is defined. When the eq matcher is used, equality is checked for a value pattern.*²

By passing a tuple matcher to a function that takes and returns a matcher, we can define a matcher for various non-free data types. For example, we can define a matcher for a graph as a set of edges. In the following code, we assume a node id is represented by an integer.

```
1 graph = multiset (integer, integer)
```

A matcher for adjacency graphs also can be defined. An adjacency graph is defined as a multiset of tuples of an integer and a multiset of integers.

```
1 adjacencyGraph = multiset (integer, multiset integer)
```

*² A definition of the eq matcher is explained in Sect. 6.3 of [3].

Some readers might wonder about matchers for algebraic data types. Egison provides a special syntax construct for defining a matcher for an algebraic data type. For example, a matcher for binary trees can be defined using `algebraicDataMatcher`.

```
1 algebraicDataMatcher binaryTree a = BLeaf a | BNode a (binaryTree a) (binaryTree a)
```

Matchers for algebraic data types and matchers for non-free data types also can be combined. For example, we can define a matcher for trees whose nodes have an arbitrary number of children whose order is ignorable. We show pattern matching for these trees in Sect. ??.

```
1 algebraicDataMatcher tree a = Leaf a | Node a (multiset (tree a))
```

第 3 章

パターンマッチ指向プログラミング 入門

- 1 リストのパターンマッチ
- 2 多重集合のパターンマッチ
- 3 複数のコレクションの比較
- 4 ループパターンを使う

第 4 章

実践パターンマッチ指向プログラミング

- 1 リストプログラミング
- 2 ポーカーの役判定
- 3 ゲーム木のパターンマッチ
- 4 N クイーン問題
- 5 SAT ソルバー

第 5 章

マッチャーを定義しよう

- 1 Egison 内部のパターンマッチアルゴリズム
- 2 `unorderedPair` マッチャーの定義
- 3 `multiset` マッチャーの定義
- 4 `soretedList` マッチャーの定義

第 6 章

Egison パターンマッチの実装

本章は，Egison パターンマッチの実装を紹介する．Egison パターンマッチの実装については，現在 2 本の論文と複数の実装が公開されている．本章はこれらの参考になる外部資料を紹介していきたい．

1 インタプリタ実装

Coq によるインタプリタ実装もある．(TODO: cite: formalized-egison)

2 ほかの関数型言語へのトランスレータ実装

3 今後の研究の展開

あとかき

2020 年 2 月 (TODO:) 日 江木 聡志

参考文献

- [1] S. Egi. Loop Patterns: Extension of Kleene Star Operator for More Expressive Pattern Matching against Arbitrary Data Structures. In *The Scheme and Functional Programming Workshop*, 2018.
- [2] S. Egi. Scheme Macros for Non-linear Pattern Matching with Backtracking for Non-free Data Types. In *The Scheme and Functional Programming Workshop*, 2019.
- [3] S. Egi and Y. Nishiwaki. Non-linear Pattern Matching with Backtracking for Non-free Data Types. In *Asian Symposium on Programming Languages and Systems*, pages 3–23. Springer, 2018.

索引

#, 5
--, 4
::, 4

matchAll, 4
matchAllDFS, 7
値パターン, 5

述語パターン, 6
パターンマッチ指向, 2
マッチャー, 4